

1-DP-Playing with Numbers

Problem Statement

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Example 1: Input: 6 Output: 6 Explanation: There are 6 ways to represent number 6: 1+1+1+1+1+1 3+3 1+1+1+3 1+1+3+1 1+3+1+1 3+1+1+1

Input Format First Line contains the number n

Output Format Print: The number of possible ways 'n' can be represented using 1 and 3

C Code

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];

        if (i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

Sample Input

6

Sample Output

6

2-DP-Playing with Chessboard

Problem Statement

Playing with Chessboard:

Ram is given an $n \times n$ chessboard with each cell with a monetary value. Ram stands at the (0,0), the position of the top left white rook. He is given a task to reach the bottom right black rook position ($n-1, n-1$) by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help Ram to achieve it by providing an efficient DP algorithm.

Example: Input 3 1 2 4 2 3 4 8 7 1

Output: 19

Explanation: The optimal path value is $1+2+8+7+1=19$.

Input Format First Line contains the integer n The next n lines contain the $n \times n$ chessboard values

Output Format Print Maximum monetary value of the path

C Code

```
#include <stdio.h>

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n][n];
    int dp[n][n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    dp[0][0] = a[0][0];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 && j == 0)
                continue;

            int up = (i > 0) ? dp[i - 1][j] : -1;
            int left = (j > 0) ? dp[i][j - 1] : -1;

            dp[i][j] = a[i][j] + max(up, left);
        }
    }

    printf("%d", dp[n - 1][n - 1]);

    return 0;
}
```

Sample Input

```
3
1 2 4
2 3 4
8 7 1
```

Sample Output

```
19
```

3-DP-Longest Common Subsequence

Problem Statement

Given two strings find the length of the common longest subsequence (need not be contiguous) between the two.

Example: s1: ggtabe s2: tgatasb

The length is 4.

Solving it using Dynamic Programming.

Example: Input: aab azb

Output: 2

C Code

```
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    char a[100], b[100];

    scanf("%s", a);
    scanf("%s", b);

    int n = strlen(a);
    int m = strlen(b);
    int dp[n + 1][m + 1];

    for (int i = 0; i <= n; i++)
        dp[i][0] = 0;

    for (int j = 0; j <= m; j++)
        dp[0][j] = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    printf("%d", dp[n][m]);

    return 0;
}
```

Sample Input

aab
azb

Sample Output

2

4-DP-Longest Non-decreasing Subsequence

Problem Statement

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Example: Input: 9 Sequence: [-1,3,4,5,2,2,2,2,3]

The subsequence is: [-1,2,2,2,2,3]

Output: 6

C Code

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n], dp[n];
    int ans = 1;

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        dp[i] = 1;
    }

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] <= a[i] && dp[j] + 1 > dp[i])
                dp[i] = dp[j] + 1;
        }

        if (dp[i] > ans)
            ans = dp[i];
    }

    printf("%d", ans);

    return 0;
}
```

Sample Input

```
9
-1 3 4 5 2 2 2 2 3
```

Sample Output

```
6
```